CS315 · ALGORITHMS ANALYSIS & DESIGN

Midterm 472 الحل

من L1 إلى L7

محارب

01 Q1: Matrix chain حل

الفكرة: عدد عمليات ضرب مصفوفة `p × q` في مصفوفة `q × r` يساوي `p·q·r`. نبي ترتيب الأقواس اللي يعطي أقل مجموع. نعبّي الجدول من السلاسل القصيرة إلى الطويلة.

```
m[i, j] = min over k of:   m[i, k] + m[k+1, j] + cost
```

```
cost = (rows of i) × (cols of k) × (cols of j)
```

1 سلاسل من مصفوفتين

```
AB  =  4 × 5 × 8  =  160
```

```
BC  =  5 × 8 × 3  =  120
```

```
CD  =  8 × 3 × 7  =  168
```

2 سلاسل من ثلاث مصفوفات

```
ABC:   A(BC)  =  0 + 120 + 4 × 5 × 3  =  180     ← min
       (AB)C  =  160 + 0 + 4 × 8 × 3  =  256
BCD:   B(CD)  =  0 + 168 + 5 × 8 × 7  =  448
       (BC)D  =  120 + 0 + 5 × 3 × 7  =  225     ← min
```

3 السلسلة كاملة ABCD

```
A(BCD)    =  0 + 225 + 4 × 5 × 7    =  365
(AB)(CD)  =  160 + 168 + 4 × 8 × 7  =  552
(ABC)D    =  180 + 0 + 4 × 3 × 7    =  264     ← min
```

	A	B	C	D
A	0	160	180	**264**
B	—	0	120	225
C	—	—	0	168
D	—	—	—	0

```
Minimum operations = 264
```

```
Best order of multiplication:   ((A(BC))D)
```

الترتيب يطلع من تتبع الاختيارات: ABCD انقسمت إلى `(ABC)D` ، و ABC انقسمت إلى `A(BC)` .

02 حل Q2: True / False + MCQ

#	الجواب	السبب
1	**True**	الحرف اللي يتكرر أكثر ياخذ code أقصر.
2	**False**	الـ greedy ياخذ أفضل اختيار حالي، وهذا ما يعطي الحل الأمثل في كل مسائل الـ DP. مثال: Matrix chain و LCS ما ينحلون greedy.
3	**False**	السلايد يكتبها `T(MERGE)` بس، والدمج يشتغل على `n` عنصر مو $\frac{n}{2}$.
4	**False**	لازم نفس البيئة **ونفس المدخلات**.
5	**True**	نص سلايد Lecture 1.
6	**c**	Dynamic programming يخزن نتائج الـ sub-problems ويستخدمها.

في رقم 6 كلمة NOT زايدة في الصياغة. المقصود «uses the past results».

03 حل Q3: LCS

نسوي جدول صفوفه حروف `X` وأعمدته حروف `Y`، مع صف وعمود أصفار في البداية. كل خانة تنحسب كذا:

```
X[i] = Y[j]:   c[i, j] = c[i-1, j-1] + 1        diagonal + 1
```

```
X[i] ≠ Y[j]:   c[i, j] = max(top, left)
```

	⌀	A	B	C	A	D	F
⌀	0	0	0	0	0	0	0
A	0	1	1	1	1	1	1
C	0	1	1	2	2	2	2
B	0	1	2	2	2	2	2
A	0	1	2	2	3	3	3
E	0	1	2	2	3	3	3
D	0	1	2	2	3	4	4

```
Length = 4
```

```
The subsequence:   1)  ABAD      2)  ACAD
```

الطول هو آخر خانة في الجدول. الحروف نطلعها بالرجوع من آخر خانة: لو الحرفين متساويين ناخذ الحرف ونطلع قطري، وإلا نروح للخانة الأكبر من فوق أو اليسار. الخانات الملونة مسار `ABAD` . لما فوق واليسار متساويين نقدر نروح لأي واحد، ومن هنا يطلع الحل الثاني `ACAD` .

```
Check:   X = A C B A E D   →   A . B A . D   and   A C . A . D
```

```
         Y = A B C A D F   →   A B . A D .   and   A . C A D .
```

04 حل Q4: Hash table

Key	4	13	49	6	40	33
k mod 9	4	4	4	6	4	6

Index	Chain
0	ø
1	ø
2	ø
3	ø
4	4 → 13 → 49 → 40
5	ø
6	6 → 33
7	ø
8	ø

أربعة مفاتيح باقي قسمتها على 9 يساوي 4، فكلها في الخانة 4 بترتيب دخولها: `4 → 13 → 49 → 40` .
والخانة 6 فيها `6 → 33` . باقي الخانات فاضية.

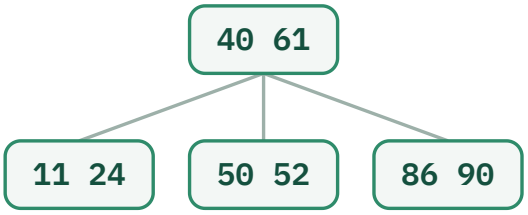
05 Q5: حل B-tree

B-tree من order 5: كل node فيها 4 مفاتيح على الأكثر و 5 أبناء على الأكثر. وكل node غير الـ root فيها مفتاحين على الأقل. لما node توصل 5 مفاتيح تنقسم (split): المفتاح الأوسط يطلع للأب، والباقي ينقسم

نصين.

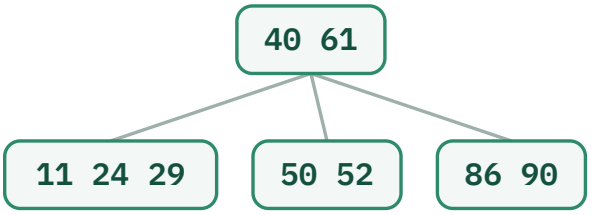
A بناء الشجرة

```
Insert 24, 52, 90, 40      →   one node    [24, 40, 52, 90]
Insert 11                  →   5 keys, split, 40 moves up
Insert 86, 61              →   right leaf    [52, 61, 86, 90]
Insert 50                  →   5 keys, split, 61 moves up
```



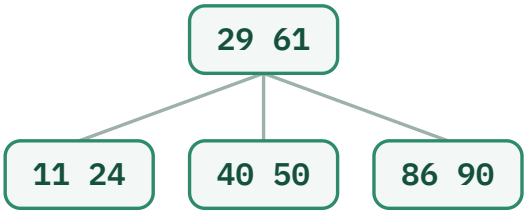
B Insert 29

`29` يروح للورقة اليسار. تصير 3 مفاتيح، ما فيه split.



Delete 52 C1

ورقة `52` يبقى فيها مفتاح واحد، أقل من الحد الأدنى (underflow). الأخ اليسار فيه 3 مفاتيح، فنستلف منه (borrow): أكبر مفتاح عنده يطلع للأب، ومفتاح الأب ينزل للورقة.



Delete 11 C2

ورقة `11` يبقى فيها مفتاح واحد. الأخ اللي جنبها فيه مفتاحين بس، فما نقدر نستلف. ندمج (merge): الورقتين مع مفتاح الأب اللي بينهم يصيرون node وحدة.

